

T.C. SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

SİSTEM PROGRAMLAMA DERSİ PROJE RAPORU

KONU: TARS AU (TEXT ARCHIVER SAKARYA UNIVERSITY) GELİŞTİRİLMESİ

GitHub: <https://github.com/kaganay/tarsau>

PROJE EKİBİ VE GÖREV DAĞILIMI

Ad Soyad	Öğrenci No	Görev Dağılımı
Kagan Aydoğan	G231210375	Mimari tasarım, -b (arşivleme) modülü, sau format motoru, Makefile, test betiği, raporlama
Kadir Duyan	G231210371	-a (açma) modülü, izin/permission yönetimi, hata mesajları, örnek metin dosyaları, dokümantasyon

1. ÖZET

Bu projede, Linux tabanlı sistemlerde çalışabilen, ASCII metin dosyalarını meta verileriyle birlikte tek bir pakette toplayan tarsau yardımcı programı geliştirilmiştir. Yazılım, verileri sıkıştırmadan paketlerken dosya isimlerini, boyutlarını ve Unix erişim izinlerini korumaktadır.

2. GELİŞTİRME SÜRECİ VE METODOLOJİ

Proje geliştirme süreci şu aşamalardan oluşmuştur:

- Format Tasarımı:** sau dosyasının header (başlık) yapısı belirlenmiş, ilk 10 baytın organizasyon boyutu için ayrılması kararlaştırılmıştır.

2. **Modüler Kodlama:** Arşivleme (-b) ve Açma (-a) işlemleri birbirini bozmayacak şekilde iki ana fonksiyonel bloğa ayrılmıştır.
3. **Versiyon Kontrolü:** Tüm süreç GitHub üzerinden yönetilmiş, ekip içi senkronizasyon git komutları ile sağlanmıştır.
4. **Otomasyon ve Test:** Hızlı derleme için Makefile, hata kontrolü için ise run_tests.sh scripti hazırlanmıştır.

3. SİSTEM MİMARİSİ VE KOD PARÇACIKLARI

3.1. İzinlerin ve Boyutun Stat ile Alınması

Dosyaların orijinal izinlerini ve boyut bilgilerini korumak için sys/stat.h kütüphanesi kullanılmıştır:

```
struct stat st; if
(stat(filenamees[i], &st) == 0) {
long filesize = st.st_size;
char perms[10];
    sprintf(perms, "%o", st.st_mode & 0777);
// İzinleri oktal formatta sakla
}
```

Bu yapı sayesinde dosyanın boyut bilgisi doğru bir şekilde okunurken, chmod değerleri (örneğin 644 veya 755) sekizlik (octal) sistemde hatasız biçimde kaydedilir.

3.2. ASCII Metin Kontrolü

Programın yalnızca ASCII karakterlerden oluşan metin dosyalarını kabul etmesi gerekliliğini sağlamak adına her dosya için karakter düzeyinde tarama gerçekleştirilir:

```
int is_text_file(const char *path) {
    FILE *f = fopen(path, "rb");
    int c;
    while ((c = fgetc(f)) != EOF) {
        if (!isascii(c)) {
            fclose(f);
            return 0;
        }
    }
    fclose(f);
    return 1;
}
```

3.3. Organizasyon Bölümü Boyutu (İlk 10 Bayt)

Arşiv yapısının en başında yer alan ve organizasyon alanının toplam büyüklüğünü belirten yapı, 10 bayt genişliğinde olacak şekilde formatlanarak dosyaya yazılır:

```
char header_len_str[11]; sprintf(header_len_str, 11,
"%010ld", total_org_size); fwrite(header_len_str, 1,
10, archive_fp);
```

3.4. Arşiv Açma ve Dinamik Dizin Oluşturma

Arşiv açma modunda (-a), dosyaların çıkartılacağı hedef dizinin sistemde var olup olmadığı kontrol edilir, yoksa dinamik olarak uygun yetkilerle oluşturulur:

```
if (access(target_dir, F_OK) == -1) {      if
(mkdir(target_dir, 0755) != 0) {
perror("Dizin olusturma hatasi");
exit(EXIT_FAILURE);      }
}
```

3.5. Dosya İzinlerinin chmod ile Uygulanması

Arşivden çıkartılan dosyaların orijinal Unix erişim yetkilerinin geri kazandırılması amacıyla, kaydedilen sekizlik izin metni sayısal değere dönüştürülerek sisteme uygulanır:

```
// Arşivden çıkarılan dosyaya orijinal izinlerini geri atama
mode_t mode = (mode_t)strtol(perm_str, NULL, 8); if
(chmod(extracted_file_path, mode) != 0) {      fprintf(stderr,
"Uyari: %s izinleri atanamadi.\n", filename); }
```

3.6. Toplam Boyut ve Dosya Sayısı Kontrolü

Uygulamanın kararlı çalışması ve ödev limitlerine bağlı kalınması amacıyla, arşivlenecek toplam dosya sayısı ve toplam boyut sınırları sıkı bir şekilde denetlenir:

```
if (total_files > 32 || total_size > (200 * 1024 * 1024)) {
fprintf(stderr, "Hata: Limitler asildi (Max 32 dosya / 200 MB).\n");
return 1;
}
```

4. UYGULAMA ÇIKTILARI (EKRAN GÖRÜNTÜLERİ)

4.1. Derleme Süreci

Makefile kullanılarak projenin derlenme aşaması terminal çıktısı:

```
kagan@Aydoan:/mnt/c/Masaüstü/G231210375_G231210371/G231210375_G231210371$ make clean &&
make rm -f tarsau demo.sau a.sau rm -rf cikti test_out test_out2 rm -f
test_files/binary.bin
gcc -Wall -Wextra -Wpedantic -O2 -std=c11 -D_POSIX_C_SOURCE=200809L -o tarsau tarsau.c
```

4.2. Arşivleme Modu (-b)

Kullanıcı iki adet metin dosyasını birleştirip arşiv oluşturduğunda ortaya çıkan terminal çıktısı:

```
kagan@Aydoan:/mnt/c/Masaüstü/G231210375_G231210371/G231210375_G231210371$ ./tarsau -b
test_files/t1.txt test_files/t2.txt -o test.sau ls -lh test.sau Toplam 2 dosya
birlestirilerek "test.sau" arshiv dosyasi olusturuldu.
-rwxrwxrwx 1 kagan kagan 369 May 16 2026 test.sau
```

4.3. Arşiv Açma Modu (-a)

Oluşturulan .sau uzantılı arşiv dosyasının hedef bir klasöre açılması ve orijinal dosya izinlerinin korunduğunun doğrulanması çıktısı:

```
kagan@Aydoan:/mnt/c/Masaüstü/G231210375_G231210371/G231210375_G231210371$ rm -rf
cikti_dizini ./tarsau -a test.sau cikti_dizini ls -l cikti_dizini "test.sau" arsivi
"cikti_dizini" dizinine acildi (2 dosya). total 0 -rwxrwxrwx 1 kagan kagan 187 May
16 2026 t1.txt
-rwxrwxrwx 1 kagan kagan 142 May 16 2026 t2.txt
```

4.4. Otomatik Test Sonuçları ve Hata Yönetimi

Programın ikili (binary) bir dosya (örneğin bir görsel) verildiğinde ASCII kontrol mekanizmasının çalışması ve hata yönetiminin çıktısı:

```
kagan@Aydoan:/mnt/c/Masaüstü/G231210375_G231210371/G231210375_G231210
371$ printf
'\x00\x01\xff' > resim.jpg ./tarsau -b resim.jpg -o hata.sau rm -f
resim.jpg resim.jpg giriş dosyasının formatı uyumsuzdur!
```

5. HATA YÖNETİMİ

Programın dayanıklılığı için aşağıdaki senaryolar kodlanmıştır:

- **Format Uyumsuzluğu:** ASCII olmayan bir dosya algılandığında program kullanıcıya "format uyumsuzdur" uyarısı verir.
- **Eksik Parametre:** Arşiv adı belirtilmezse varsayılan olarak a.sau kullanılır.
- **Bozuk Arşiv:** Header yapısı bozuk dosyalarda güvenli çıkış yapılır.
- **Limit Aşımı:** 32 dosyadan fazla veya toplamda 200 MB'ı geçen isteklerde işlem durdurularak standart hataya bildirim basılır.

6. SONUÇ

Geliştirilen tarsau uygulaması, sistem programlama dersinin gereksinimlerini tam olarak karşılamaktadır. Ekip çalışması sayesinde hem backend motoru hem de kullanıcı arayüzü ve hata yönetimi kısımları optimize edilmiş, Linux standartlarına uygun bir arşivleyici ortaya çıkmıştır.

7. KAYNAKLAR

1. The C Programming Language (Kernighan & Ritchie)
2. GNU/Linux System Calls Reference
3. SAU Bilgisayar Mühendisliği Sistem Programlama Ödev Talimatları